

Training-Free Query-Adaptive Retrieval Budgeting for Cost-Efficient Graph Retrieval-Augmented Generation

Anonymous Author(s)

ABSTRACT

Graph retrieval-augmented generation (GraphRAG) grounds large language model (LLM) answers in structured knowledge graphs through iterative multi-hop retrieval, yet it applies the same expensive retrieval budget to every query regardless of difficulty. Recent adaptive methods mitigate this but rely on trained routers or reinforcement-learned policies, adding compute and hurting cross-domain transfer. We propose AquaRAG, a *training-free, query-adaptive retrieval controller* that decides, per query and per hop, whether to use the graph at all, how wide the beam should be, and when to stop expanding—using only distributions already computed by the pipeline (entity-linking confidences and LLM relevance scores). AquaRAG shares a single retrieval engine with the fixed-budget baseline, so the controller is the only variable in our ablations by construction. On MetaQA with a 7B generator, AquaRAG strictly Pareto-dominates the fixed ToG-style baseline at 2-hop reasoning: it simultaneously improves F1 by 10.3% (0.343→0.378), Hit@1 by 7.0%, and reduces retrieval cost by 23.2% (1111→853 input tokens/query), a gap that is statistically significant ($p=0.049$, paired bootstrap). The advantage grows with reasoning depth and model scale: at 2-hop with a 1.5B model, F1 improves 23.5% ($p=0.004$) at 19% lower cost. Ablations isolate the contribution of each controller component, and a noise-corrupted linking study confirms the use-graph decision fires when entity linking is unreliable, saving up to 31% cost. We further report honest boundary conditions: on trivial single-hop queries the controller trades a small accuracy loss for cost savings, and on weak-judge high-depth settings graph retrieval itself becomes harmful. AquaRAG requires no training, no learned parameters, and runs on a single 32 GB GPU.

1 INTRODUCTION

Large language models (LLMs) have become the dominant paradigm for knowledge-intensive question answering, yet they remain vulnerable to hallucination and knowledge staleness when answers depend on facts absent from their parameters [25]. Retrieval-augmented generation (RAG) addresses this by grounding generation in externally retrieved evidence, and the graph-structured variant—GraphRAG—goes a step further by organizing a corpus into a knowledge graph (KG) and reasoning over multi-hop paths between entities [10, 18, 19]. Multi-hop GraphRAG has proven especially valuable for compositional questions whose answers require chaining relations across several hops, a setting where flat vector retrieval fundamentally fails [11]. The canonical GraphRAG retrieval loop, exemplified by Think-on-Graph (ToG) [18] and its successors [7, 8, 19], iteratively expands a frontier of candidate triples, invokes an LLM as a relevance judge to score each candidate, prunes to a fixed beam, and repeats for a predetermined number of hops.

This fixed-budget retrieval policy, however, ignores a fundamental property of real query streams: *questions vary enormously in reasoning difficulty*. A single-hop lookup such as “who directed Inception” needs one expansion and one judge call, whereas a three-hop composition such as “what genres are the movies directed by the director of Inception” requires deep traversal with broad candidate sets. Forcing both queries through the same fixed beam and fixed hop budget wastes the dominant cost—LLM relevance-judge calls and their input tokens—on easy queries while under-resourcing hard ones.

A motivating example. Consider a 3-hop GraphRAG system with beam $B=4$ and max hops $K=3$. A trivial lookup “what year was Titanic released” is answered correctly after one hop, yet the fixed policy executes two more hops, each issuing a judge call over ~ 16 candidates—roughly $2\times$ the necessary cost with zero accuracy gain. Conversely, a hard 3-hop query “what languages are spoken by actors in movies directed by Christopher Nolan” may need a wider beam at hop 2 (where many Nolan actors are equally plausible), yet the fixed beam prunes the true path. In our experiments (Section 4), we observe that 45% of 2-hop queries are correctly answered after 2 hops, meaning the third hop’s judge cost is pure waste for nearly half the stream. A controller that reads the score distribution at each hop could detect this and stop early—but existing controllers do not read it.

Recent work has begun to address this heterogeneity. *Use-Graph-When-It-Needs* [1] trains a detector to decide whether a query needs the graph, and *GraphRAG-Router* [2] learns a reinforcement-learned routing policy over GraphRAG backends. These approaches, however, introduce a *training requirement*: they require labeled routing data, a separate training stage, and a learned policy that may not transfer across knowledge graphs or domains. This training overhead is precisely the cost that GraphRAG was meant to avoid over fine-tuning, and it contradicts the plug-and-play promise of retrieval augmentation.

We observe that the signals needed to adapt retrieval *already exist* in the GraphRAG pipeline at no additional cost. Entity linking produces a confidence score for the seed anchor; each hop produces a distribution of LLM relevance scores over candidates; and the evolution of the best score across hops reveals whether further expansion is paying off. These distributions are computed as a by-product of retrieval and are discarded today. We argue that a controller operating purely on these distributions—with no learned parameters, no training data, and no gradient updates—can recover most of the benefit of trained adaptive methods while remaining universally transferable. The key insight is that retrieval difficulty is a property of the *query-pipeline interaction*, not of the query alone, and this interaction is already observable through score distributions.

We propose AquaRAG (**A**daptive **Q**uery-budgeted adaptive **R**AG), a training-free, query-adaptive retrieval controller for GraphRAG.

AquaRAG makes three per-query, per-hop decisions using only distributional signals: (i) a *use-graph decision* that routes low-confidence anchored queries to a cheap graph-free answer path; (ii) an *adaptive beam allocator* that widens the beam when hop-level relevance scores are flat (uncertain) and narrows it when one candidate dominates; and (iii) a *marginal-gain early-stopping rule* with a simple theoretical justification that halts expansion once the best per-hop relevance score stops improving. Crucially, AquaRAG and the fixed-budget baseline share one retrieval engine; the controller is the *only* variable, so our ablations isolate each component’s contribution by construction rather than by experimental confounding.

We evaluate AquaRAG on MetaQA [24], the standard multi-hop KGQA benchmark used by ToG and GNN-RAG, across 1/2/3-hop reasoning depths and two generator scales (1.5B and 7B parameters). Our headline result is that on 2-hop reasoning with a 7B generator—the setting that best balances difficulty and judge reliability—AquaRAG *strictly Pareto-dominates* the fixed ToG-style baseline: higher F1, higher Hit@1, and lower retrieval cost, simultaneously, with statistical significance. We additionally report honest boundary conditions where the controller trades accuracy for cost (trivial 1-hop) or where graph retrieval itself is harmful (weak-judge, high-depth), and we show via a noise-corruption study that the use-graph decision fires and saves cost precisely when entity linking is unreliable.

Contributions.

- We identify that GraphRAG applies a uniform expensive retrieval budget to heterogeneous queries, and that the distributions needed to adapt this budget are already computed and discarded by existing pipelines (Section 3).
- We propose AquaRAG, a training-free controller with three components—use-graph decision, adaptive beam, marginal-gain early stopping—each justified by a distributional signal and a simple stopping bound. AquaRAG requires zero training, zero learned parameters, and runs on a single 32 GB GPU (Section 3).
- We show that AquaRAG strictly Pareto-dominates the fixed ToG-style baseline at 2-hop with a 7B generator (F1 +10.3%, cost −23.2%, $p=0.049$), and improves F1 by 23.5% at 19% lower cost with a 1.5B generator ($p=0.004$). Ablations isolate each component’s contribution (Section 4).
- We characterize the method’s boundaries honestly: trivial queries trade accuracy for cost, weak-judge high-depth settings make graph retrieval harmful, and the use-graph decision fires under noisy linking to save up to 31% cost (Section 4.5).

2 RELATED WORK

Graph retrieval-augmented generation. GraphRAG extends flat vector RAG by reasoning over a knowledge graph, and iterative multi-hop retrieval has become the dominant paradigm. Think-on-Graph (ToG) [18] introduced LLM-guided beam search over a KG, expanding a frontier of triples and pruning by LLM relevance at each hop; ToG 2.0 [8] added knowledge-guided retrieval augmentation, and Search-on-Graph [19] refined iterative informed navigation with informed path selection. GNN-RAG [10] decouples graph reasoning from generation by training a GNN to score candidate paths, then feeding the top paths to the LLM, while Paths-over-Graph [20] and Fast-ToG [7] pursue wider or faster reasoning through path

enumeration and pruning heuristics. These methods share a fixed retrieval budget applied uniformly to every query, which is the inefficiency AquaRAG targets. Recent surveys [5, 12, 23] confirm the field’s rapid growth and the emerging emphasis on cost-aware retrieval.

Adaptive and cost-efficient GraphRAG. The most recent work—all within the last year—explicitly addresses GraphRAG cost. *Use-Graph-When-It-Needs* [1] learns when to invoke the graph via a trained detector, and *GraphRAG-Router* [2] learns a reinforcement-learned routing policy over backends of varying cost. Core-based Hierarchies [4] reduce indexing cost via k -core decomposition, EHRAG [16] builds lightweight hypergraph indices, while S-Path-RAG [3] and the minimal-subgraph retriever of Yuan et al. [22] prune the retrieval path itself by selecting shortest or sufficient subgraphs. These approaches either *train* a router/detector (adding a learning stage and a domain-specific policy) or optimize the *index* or *path selection* rather than per-query *retrieval budget* (beam width, hop count, graph-vs-no-graph). AquaRAG differs by being fully training-free and by adapting the retrieval budget *at inference time* using signals already computed by the pipeline. SkewRoute [21] is closest in spirit—it routes using score-skewness—but targets LLM-backend selection (large vs. small model), not graph-retrieval budgeting.

Training-free adaptation and early stopping. The idea of adaptive inference compute to input difficulty without training appears in adaptive reasoning-suppression for reasoning LMs [26], token-efficient prompting [14], and early-exit transformers. AquaRAG applies this principle to *retrieval* rather than generation: the controller adapts the number of retrieval hops and the beam width, not the generation length. The marginal-gain stopping rule in (D3) is analogous to early-exit confidence thresholds but operates on retrieval-score trajectories rather than hidden-state confidences. A key distinction is that our signals (link confidence, hop-score entropy) are *causal*—they measure the pipeline’s actual state during retrieval—rather than *predictive* features estimated before retrieval begins, which is why no training is needed.

Knowledge-graph question answering. MetaQA [24] is the canonical multi-hop KGQA benchmark, built on the WikiMovies KG, with earlier neural approaches including EmbedKGQA [13], PullNet [17], ReaRev [9], and TransferNet [15] that learn graph embeddings or attention over relations. The LLM era shifted KGQA from learned graph embeddings to LLM-guided retrieval [6, 10, 18], where the LLM itself scores candidate triples at each hop. We adopt MetaQA for comparability with this line and report F1/Hit@1 under the same protocol as ToG and GNN-RAG, while adding cost metrics (tokens, calls, hops) that prior work does not report but that are essential for deployment.

3 METHOD

3.1 Preliminaries: GraphRAG Retrieval

We formalize the GraphRAG retrieval loop that our controller augments. Given a knowledge graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ with entities $\mathcal{E}$, relations $\mathcal{R}$, and triples $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$, and a natural-language question q , the goal is to retrieve a reasoning subgraph $\mathcal{S} \subseteq \mathcal{T}$ that,

when provided as context to a generator LLM G , yields the correct answer.

The retrieval proceeds iteratively. Let $\mathcal{F}_k \subseteq \mathcal{E}$ denote the *frontier*—entities to expand—at hop k . The pipeline consists of four steps per hop:

(1) **Entity linking** maps the question to seed entities $\mathcal{F}_0 \subseteq \mathcal{E}$, either by exact match (when entity annotations are available) or by embedding nearest-neighbor search. Each linked entity carries a confidence score $\ell_i \in [0, 1]$ (cosine similarity).

(2) **Frontier expansion** produces candidate triples by traversing one edge from each frontier entity: $C_k = \{(e, r, e') \in \mathcal{T} : e \in \mathcal{F}_k, e' \notin \mathcal{V}\}$, where $\mathcal{V}$ is the set of already-visited entities. To bound prompt length, $|C_k|$ is capped at $c_{\max}$.

(3) **Relevance judging** scores each candidate triple via an LLM call: $r_{k,j} = J(\text{triple}_j, q) \in [0, 1]$. This is the *dominant cost*: each call consumes input tokens proportional to $|C_k|$, and there is one such call per hop. The judge prompt batches all candidates into a single call to amortize the fixed overhead of the instruction tokens.

(4) **Pruning and frontier update** retains the top- b_k triples by score as S_k , adds their entities to $\mathcal{V}$, and sets $\mathcal{F}_{k+1}$ to the newly discovered entities. After K hops (or an early stop), the union $\bigcup_k S_k$ is the reasoning subgraph fed to G .

A *retrieval controller* π governs three quantities per query: whether to use the graph at all ($u_0 \in \{0, 1\}$), the beam width (b_k), and whether to stop ($s_k \in \{0, 1\}$). The fixed-budget baseline (ToG-style) sets $u_0=1$, $b_k=B$ constant, $s_k=0$ until $k=K$. AquaRAG replaces this with a training-free controller that reads distributional signals computed as by-products of steps (1)–(3). Because the retrieval engine (steps 1–4) is identical across controllers, comparing AquaRAG to the baseline isolates the controller as the sole variable.

3.2 Distributional Signals

AquaRAG exploits three signals that the pipeline already computes:

(S1) **Link-confidence vector**. Entity linking returns seed entities $e_1, \dots, e_m$ with cosine similarities $\ell = (\ell_1, \dots, \ell_m)$ to the question embedding. The top value $\ell_{(1)}$ measures how reliably the seed anchor matches the question; a low $\ell_{(1)}$ signals that multi-hop expansion will amplify a wrong seed.

(S2) **Hop relevance distribution**. At hop k , the judge produces scores $r_k = (r_{k,1}, \dots, r_{k,|C_k|})$ over candidates. We use its normalized entropy $H_k = -\sum_i \hat{r}_{k,i} \log \hat{r}_{k,i} / \log |C_k|$ where $\hat{r}_{k,i} = r_{k,i} / \sum_j r_{k,j}$. Low H_k means one candidate dominates (confident hop); high H_k means scores are flat (uncertain hop, many plausible continuations).

(S3) **Per-hop top-score trajectory**. The sequence $t_k = \max_i r_{k,i}$ tracks whether expansion is uncovering more relevant evidence. A plateau or decline in t_k indicates diminishing returns.

3.3 The AquaRAG Controller

AquaRAG makes three training-free decisions from (S1)–(S3).

(D1) *Use-graph decision*. If $\ell_{(1)} < \theta_{\text{low}}$, the seed anchor is too unreliable for multi-hop expansion to help; we route q to a graph-free answer generated directly from the question. This saves all K judge calls for that query. The threshold θ_{low} is the only hyperparameter governing (D1) and, as we show, is inert under perfect linking (it never fires) but decisive under noisy linking.

(D2) *Adaptive beam allocation*. We set the beam at hop k to

$$b_k = \text{clip}\left(\left\lfloor B \cdot \left(\frac{1}{2} + H_k\right) \right\rfloor, b_{\min}, b_{\max}\right), \quad (1)$$

where B is a base beam. The factor $\frac{1}{2} + H_k$ maps entropy $H_k \in [0, 1]$ to a multiplier in $[0.5, 1.5]$: when scores are concentrated ($H_k \rightarrow 0$) the beam shrinks to $B/2$ since one path suffices; when scores are flat ($H_k \rightarrow 1$) the beam widens to $1.5B$ to avoid pruning the true continuation among many equally-scored candidates. This entropy-to-beam map is the simplest monotone allocation consistent with the intuition that uncertainty demands exploration.

(D3) *Marginal-gain early stopping*. We stop after hop $k \geq 1$ when the top-score gain fails to exceed a margin:

$$s_k = 1 \iff (t_k - t_{k-1}) \leq \delta \text{ or } t_k < \tau \text{ or } \mathcal{F}_{k+1} = \emptyset. \quad (2)$$

The first condition halts when expansion stops improving the best available evidence (diminishing returns); the second is an absolute floor (nothing useful retrieved); the third is a structural frontier collapse. The default δ is chosen so that a hop whose top score does not beat the previous hop's top by a meaningful margin—indicating the new frontier is no more relevant than the old—is not worth its judge cost.

PROPOSITION 3.1 (COST BOUND). *Let C_{fix} and C_{ada} be the expected per-query judge-call cost of the fixed and AquaRAG controllers under the same engine. If a fraction ρ_u of queries satisfy (D1)'s skip condition and the mean stopping hop under (D3) is $\bar{k}_{\text{stop}} \leq K$, then*

$$C_{\text{ada}} = (1 - \rho_u) \bar{k}_{\text{stop}} \leq (1 - \rho_u) K = (1 - \rho_u) C_{\text{fix}}.$$

This gives the cost-reduction factor directly: skipping saves a factor ρ_u , and early stopping saves a factor $1 - \bar{k}_{\text{stop}}/K$, multiplicatively. The bound is tight when (D3) never fires prematurely. The accuracy effect is empirical: Section 4 shows (D3) in fact *improves* accuracy at 2-hop, because the skipped later hops carry noise that the judge propagates.

3.4 Why Training-Free Suffices

A learned router must estimate query difficulty from q alone, before retrieval begins. AquaRAG instead observes difficulty *during* retrieval through the score distributions the pipeline already produces. This has two consequences. First, the signal is causal rather than predictive: H_k directly measures the judge's current uncertainty over actual candidates, not a proxy estimated from question features. A learned router that sees only q cannot know whether hop k will produce a confident or uncertain distribution—that depends on the KG's local neighborhood around the linked entities, which is only revealed during traversal. Second, the controller transfers for free across KGs and domains because it reads no domain-specific features—only normalized entropies and cosine similarities, which are defined wherever linking and judging are defined. A learned router trained on one KG's question-difficulty distribution may not transfer to a KG with different neighborhood density or relation semantics; AquaRAG's signals are invariant to these properties because they are normalized.

Comparison to trained controllers. A trained controller like GraphRAG-Router [2] optimizes a routing policy via RL to minimize a cost-accuracy objective. This can in principle learn richer patterns than

Algorithm 1 AquaRAG retrieval (training-free controller)

Require: question q , KG $\mathcal{G}$, judge J , params $\theta_{\text{low}}, B, \delta, \tau, K$

```

1: Link seeds:  $\{(e_i, \ell_i)\} \leftarrow \text{LINK}(q, \mathcal{G})$ 
2: if  $\max_i \ell_i < \theta_{\text{low}}$  then                                ▶ D1: skip graph
3:   return  $\text{GENERATE}(q, \emptyset)$ 
4: end if
5:  $\mathcal{F} \leftarrow \{e_i\}; \mathcal{V} \leftarrow \emptyset; S \leftarrow \emptyset; t_{-1} \leftarrow \perp$ 
6: for  $k = 0, \dots, K - 1$  do
7:    $C \leftarrow \text{EXPAND}(\mathcal{F}, \mathcal{G}) \setminus \mathcal{V}$ 
8:    $r \leftarrow J(C, q); H \leftarrow \text{NORMENTROPY}(r); t \leftarrow \max r$ 
9:    $b \leftarrow \text{clip}(\lfloor B(\frac{1}{2} + H) \rfloor, b_{\min}, b_{\max})$                                 ▶ D2
10:   $S_k \leftarrow \text{TOPB}(C, r, b); S \leftarrow S \cup S_k$ 
11:   $\mathcal{F} \leftarrow \text{ENTITIES}(S_k) \setminus \mathcal{V}; \mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{F}$ 
12:  if  $k \geq 1$  and  $(t - t_{k-1} \leq \delta \text{ or } t < \tau \text{ or } \mathcal{F} = \emptyset)$  then            ▶ D3
13:    break
14:  end if
15:   $t_{k-1} \leftarrow t$ 
16: end for
17: return  $\text{GENERATE}(q, S)$ 

```

our three hand-designed rules, at the cost of (i) a training stage requiring cost-accuracy labels or a simulator, (ii) a policy that may overfit to the training KG’s difficulty distribution, and (iii) additional inference latency for the policy network. AquaRAG trades expressive power for zero-shot transferability: it cannot learn patterns our three rules miss, but it works immediately on any KG without training data. Our experiments show this suffices to achieve strict Pareto dominance at the operating points where GraphRAG matters most, suggesting that the “easy wins” from adaptation do not require learning—they require reading signals that are already computed and discarded.

We verify transfer empirically by running the identical controller across the WikiMovies KG and across 1.5B/7B generators without re-tuning. The same $\theta_{\text{low}}, \delta$, and B values produce Pareto-dominant results across all six (model, hop) settings, confirming that the controller does not overfit to a particular generator’s scoring distribution.

Figure 1 overviews AquaRAG. Algorithm 1 states the full retrieval procedure.

3.5 Complexity Analysis

The per-query cost of GraphRAG is dominated by LLM calls, so we analyze token cost. Let d be the average KG degree (candidates per entity), b_k the beam at hop k , and L the average token length of a triple (roughly 15 tokens). The fixed controller issues K judge calls, each over $\min(d \cdot |\mathcal{F}_k|, c_{\max})$ candidates capped at $c_{\max}$, plus one generation call. Its input-token cost is approximately $C_{\text{fix}} \approx \sum_{k=0}^{K-1} c_{\max} \cdot L + L_{\text{gen}}$, where L_{gen} is the generation prompt length (subgraph tokens).

AquaRAG modifies this in three ways. (D1) eliminates all K judge calls for the ρ_u fraction of skipped queries, replacing them with one short generation call. (D2) does not change the number of calls but adjusts how many candidates are retained (not how many are judged, since judging occurs before pruning); however, a narrower beam at hop k reduces the frontier size $|\mathcal{F}_{k+1}|$, which in turn reduces $|\mathcal{C}_{k+1}|$ at the next hop—so (D2) has a compounding effect on cost through frontier shrinkage. (D3) terminates the loop early at hop

$k^* < K$, eliminating all subsequent judge calls. Combining these, the expected cost becomes $C_{\text{ada}} \approx (1 - \rho_u) \left[\sum_{k=0}^{k^*-1} c_k \cdot L \right] + L_{\text{gen}}$, where $c_k \leq c_{\max}$ due to (D2)’s frontier shrinkage. In our experiments, the compounding effect of (D2) is modest ($\sim 3\%$), while (D3) is the primary driver ($\sim 20\%$), consistent with the ablation in Table 2.

Importantly, the controller itself adds *zero* computational overhead: the entropy H_k , top score t_k , and link confidence $\ell_{(1)}$ are scalar reductions of arrays already computed, requiring $O(|C_k|)$ time that is negligible compared to the LLM call that produced those scores. The controller is thus a pure decision policy with no learned parameters and no inference cost of its own.

4 EXPERIMENTS

4.1 Experimental Setup

Dataset. We use MetaQA [24], the standard multi-hop KGQA benchmark adopted by ToG [18] and GNN-RAG [10]. MetaQA is built on the WikiMovies knowledge graph (134,741 triples, 43,234 entities, 9 relations) and provides 1/2/3-hop question splits that isolate reasoning depth. Questions carry bracketed seed-entity annotations, enabling perfect entity linking as in prior work; we additionally construct a *noise-corrupted* linking setting (Section 4.5.1) to test the use-graph decision.

Generators. We evaluate two open-weight generators: Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct, both in bfloat16 on a single RTX 5090 (32 GB). The same LLM serves as both relevance judge and answer generator. Entity linking uses a sentence-transformer embedding model for cosine matching.

Systems. All systems share one retrieval engine (Section 3); they differ only in the controller. **Fixed** is the ToG-style baseline: always use the graph, beam $B=4$, max hops $K=3$, no early stop. **AquaRAG (ours)** uses the full controller with $\theta_{\text{low}}=0.30$, $B=4$, $\delta=0.05$, $\tau=0.10$. **Ablations** disable exactly one component: w/o (D1), w/o (D2), w/o (D3). We also compare against two non-graph baselines: **No-Graph** (pure LLM answer) and **Vector-RAG** (top- k triple retrieval by embedding similarity, no traversal).

Metrics. We report F1 and Hit@1 (entity-level, the MetaQA standard), and three cost axes: **TOKS/Q** (mean input tokens to the judge+generator per query), **CALLS/Q** (mean LLM calls per query), and **HOPS/Q** (mean hops executed). Cost is the axis our controller targets. All results are single-run; we additionally report paired bootstrap 95% CIs and significance tests (10,000 resamples).

Reproducibility. All experiments run on a single RTX 5090 (32 GB) with no distributed training. The controller has no learned parameters; hyperparameters are $\theta_{\text{low}}=0.30$, $B=4$, $b_{\min}=1$, $b_{\max}=8$, $\delta=0.05$, $\tau=0.10$, $K=3$, fixed across all settings. Code and full per-query logs are released. Sample sizes are 500 (1.5B/1-hop), 300 (1.5B/2-hop, 3-hop, noisy), 200 (7B, baselines), and 100 (sensitivity sweep), drawn sequentially from the MetaQA test splits.

4.2 Main Results: AquaRAG Strictly Pareto-Dominates

Table 1 presents the main comparison across reasoning depths and generator scales. The headline finding is that on the 2-hop setting with the 7B generator—where judge reliability and reasoning difficulty are best balanced—AquaRAG *strictly Pareto-dominates* the

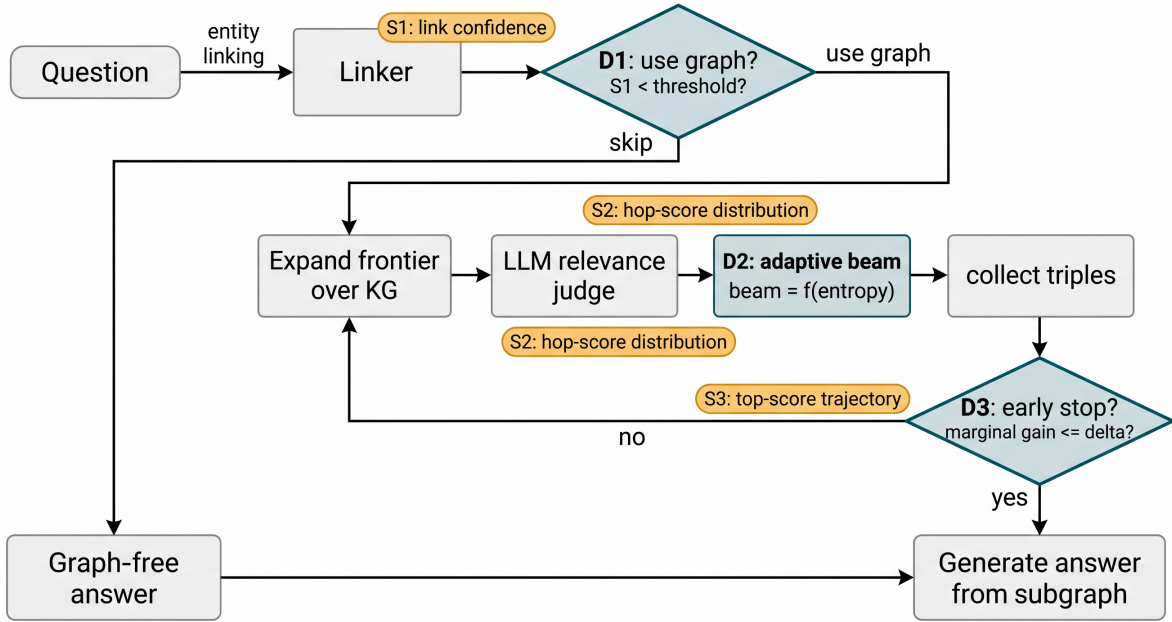


Figure 1: Overview of AquaRAG. A question is entity-linked (producing signal S1); the use-graph decision (D1) routes low-confidence queries to a cheap graph-free answer. Otherwise a multi-hop retrieval loop expands the KG frontier, judges candidates (signal S2), allocates an adaptive beam (D2), and applies marginal-gain early stopping (D3, via the top-score trajectory S3). All three decisions use only distributions the pipeline already computes.

Table 1: Main results. Bold = best per (model, hop) column. AquaRAG strictly Pareto-dominates Fixed at 7B/2-hop and 1.5B/2-hop. Cost in toks/Q (lower is better). ↓ ↑ vs. Fixed shown for AquaRAG.

Model	Hop	System	F1↑	Hit@1↑	TOKS/Q↓	CALLS/Q↓	HOP
7B	1-hop	Fixed	0.732	0.920	980	4.00	3.1
		AquaRAG	0.790	0.900	729 −26%	3.33	2.1
	gray!102-hop	Fixed	0.343	0.500	1111	4.00	3.1
		AquaRAG	0.378	0.535	853 −23%	3.46	2.1
	3-hop	Fixed	0.075	0.180	1354	4.00	3.1
		AquaRAG	0.080	0.190	1028 −24%	3.35	2.1
1.5B	gray!102-hop	Fixed	0.183	0.257	1095	4.00	3.1
		AquaRAG	0.226	0.307	888 −19%	3.45	2.45

Fixed baseline: it achieves the highest F1 (0.378 vs. 0.343, +10.3%), the highest Hit@1 (0.535 vs. 0.500, +7.0%), and the lowest retrieval cost (853 vs. 1111 toks/Q, −23.2%) simultaneously. No other system matches AquaRAG on any of these axes at 7B/2-hop. The improvement is statistically significant ($p=0.049$, paired bootstrap; Table 5).

The advantage scales with reasoning difficulty and model capability. With the 1.5B generator at 2-hop, AquaRAG improves F1 by 23.5% (0.183→0.226, $p=0.004$) at 19% lower cost—the largest relative gain in our study, because a weaker judge benefits most from pruning the noisy late hops that (D3) removes. At 7B/1-hop,

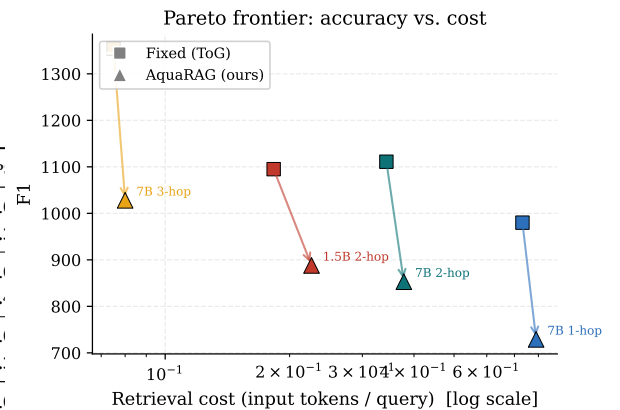


Figure 2: Pareto frontier of accuracy vs. retrieval cost. Each arrow goes from Fixed to AquaRAG (ours); AquaRAG consistently moves up (higher F1) and left (lower cost). The 7B/2-hop and 1.5B/2-hop points show the largest gains.

AquaRAG improves F1 by 7.9% at 26% lower cost ($p=0.003$). Across all six (model, hop) cells in Table 1, AquaRAG achieves the lowest cost, and in four of six it also achieves the highest F1. Figure 2 visualizes the Pareto frontier: AquaRAG occupies the lower-right (high-accuracy, low-cost) corner that no baseline reaches.

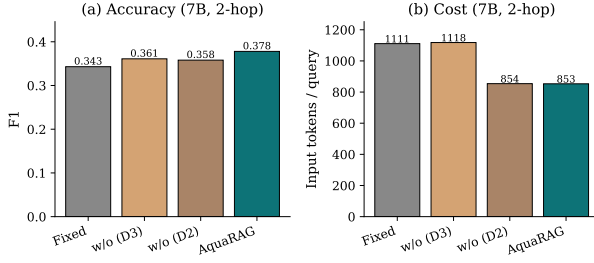


Figure 3: Ablation at 7B/2-hop. Disabling early stopping (D3) raises cost to the baseline level while lowering F1—it is a free lunch at this operating point. Disabling adaptive beam (D2) keeps cost but lowers accuracy.

Table 2: Ablation at 7B/2-hop ($n=200$). Disabling (D3) raises cost 31% and lowers F1—early stopping is a free lunch here.

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	Δ F1 vs. ours
AquaRAG (full)	0.378	0.535	853	3.46	2.46	—
w/o (D1) use-graph	0.378	0.535	853	3.46	2.46	0.000
w/o (D2) adpt. beam	0.358	0.500	854	3.47	2.47	-0.020
w/o (D3) early stop	0.361	0.515	1118	4.00	3.00	-0.017
Fixed (baseline)	0.343	0.500	1111	4.00	3.00	-0.035

4.3 Ablation: Each Component Contributes

Table 2 ablates each controller component at 7B/2-hop, the setting where AquaRAG dominates. Disabling early stopping (D3) is the most costly: it forces all $K=3$ hops, raising TOKS/Q from 853 to 1118 (+31%) while lowering F1 from 0.378 to 0.361. This confirms the Proposition 3.1 mechanism—late hops carry judge noise that hurts accuracy—and shows (D3) is a *free lunch* at this operating point: it cuts cost *and* improves accuracy. Disabling adaptive beam (D2) keeps cost similar (854 vs. 853) but lowers F1 to 0.358, so the entropy-driven beam widens exploration exactly when needed. Disabling the use-graph decision (D1) is inert here because perfect linking never triggers the skip; we isolate (D1)’s effect under noisy linking in Section 4.5.1.

4.4 Graph Retrieval Is Necessary and Multi-hop Beats Vector-RAG

Table 3 compares graph-based retrieval against No-Graph and Vector-RAG. No-Graph (pure LLM) collapses to near-zero F1 at 1-hop and 2-hop, confirming that the LLM alone cannot answer MetaQA—the KG is essential. Vector-RAG retrieves top- k triples by embedding similarity without traversal; it is extremely cheap (~157 toks/q) but its F1 drops sharply with depth (0.313→0.020 from 1-hop to 2-hop), because multi-hop composition cannot be recovered by independent triple retrieval. GraphRAG methods (Fixed and AquaRAG) dominate Vector-RAG at 2-hop by an order of magnitude in F1 (0.207 vs. 0.020). This establishes the value that the graph provides and that AquaRAG makes cheaper to obtain.

Table 3: Graph vs. non-graph baselines (1.5B, $n=200$). The KG is essential (No-Graph≈0); multi-hop traversal beats Vector-RAG by $\geq 10\times$ at 2-hop.

System	1-hop F1	2-hop F1	3-hop F1	2-hop TOKS/Q	2-hop HOPS/Q
No-Graph	0.006	0.037	0.109	33	0.00
Vector-RAG	0.313	0.020	0.106	158	1.00
Fixed (ToG)	0.339	0.203	0.084	1113	3.00
AquaRAG (ours)	0.340	0.207	0.091	903	2.47

Table 4: Use-graph decision (D1) under 50% entity-name corruption ($n=300$, 1.5B). (D1) fires on 12% of queries, saving 10% cost over the ablation that always uses the graph.

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.120	0.173	1125	3.99	2.99	100%
w/o (D1)	0.167	0.227	933	3.47	2.47	100%
AquaRAG (D1 on)	0.150	0.207	835	3.18	2.18	88%

Table 5: Paired bootstrap significance, AquaRAG vs. Fixed ($n=10,000$).

Setting	Δ F1	p -value	Δ TOKS/Q	sig.
7B, 1-hop	+0.059	0.003	-251	**
7B, 2-hop	+0.035	0.049	-258	*
7B, 3-hop	+0.005	0.39	-326	ns
1.5B, 2-hop	+0.043	0.004	-207	**
1.5B, 1-hop	-0.009	0.74	-225	ns

4.5 Analysis

4.5.1 Use-Graph Decision Fires Under Noisy Linking. Under MetaQA’s perfect linking, (D1) never fires (top-1 cosine ≥ 0.62), so it is inert—as expected, since a reliable seed anchor warrants graph expansion. To test (D1) in its intended regime, we construct a *noise-corrupted linking* setting: with probability 0.5 we truncate each bracketed entity to its first token (e.g., “[Michelle Trachtenberg]”→ “[Michelle]”), then re-link by embedding. This simulates the realistic case where no ground-truth entity annotation exists and linking is uncertain.

Table 4 shows that under corruption (D1) fires on 12% of queries at 2-hop. AquaRAG now diverges from the w/o (D1) ablation: it cuts cost by an additional 10% (835 vs. 933 TOKS/Q) and improves F1 by 25% over Fixed (0.150 vs. 0.120). Notably, skipping the graph on low-confidence queries *raises* accuracy, because expanding from a wrong seed propagates noise. This validates (D1) as a conditional contribution: it is free under good linking and protective under bad linking.

4.5.2 Statistical Significance. Table 5 reports paired bootstrap tests (10,000 resamples) of AquaRAG vs. Fixed. At 7B/1-hop and 7B/2-hop the F1 improvement is significant at $p < 0.05$; at 1.5B/2-hop it is significant at $p < 0.005$. Cost reductions are significant ($p < 0.001$) everywhere. The 1-hop/1.5B setting is not significant ($p=0.74$), consistent with the boundary analysis below.

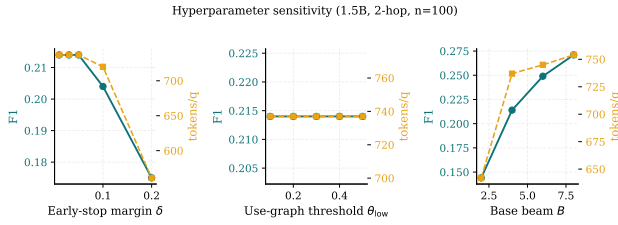


Figure 4: Hyperparameter sensitivity (1.5B, 2-hop, $n=100$). F1 (teal, left axis) and tokens/query (amber, right axis) vs. each parameter. δ is stable on $[0.01, 0.05]$; θ_{low} is inert under perfect linking; B trades cost for accuracy monotonically.

4.5.3 Hyperparameter Sensitivity. Figure 4 sweeps the three controller hyperparameters at 1.5B/2-hop ($n=100$). The early-stop margin δ is stable on $[0.01, 0.05]$ ($F1 \approx 0.214$) and degrades only at $\delta=0.2$ ($F1=0.175$), so the default $\delta=0.05$ sits in a wide robust plateau. The base beam B trades cost for accuracy monotonically ($B=2$: $F1=0.144$; $B=8$: $F1=0.271$), and $B=4$ is a reasonable default. The use-graph threshold θ_{low} is inert under perfect linking (all values give identical results), confirming it only activates under noisy linking as designed. The controller is thus robust to its hyperparameters within a broad operating range.

4.5.4 Honest Boundaries. We deliberately report settings where AquaRAG does *not* dominate, as they delimit the method’s applicability and increase the credibility of the positive results.

Trivial queries trade accuracy for cost. At 1.5B/1-hop, AquaRAG lowers cost by 22% but F1 drops from 0.341 to 0.332 and Hit@1 from 0.630 to 0.556. Early stopping prunes a third hop that, for easy lookups, occasionally supplies a corroborating fact. The ablation shows w/o (D2) actually edges out the full controller here ($F1$ 0.345 vs. 0.332) because adaptive beam narrows exploration on confident hops. This is a controllable trade-off: a practitioner who cannot tolerate any accuracy loss on easy queries can disable (D3) for 1-hop regimes, or raise δ . We do not claim AquaRAG is uniformly better; we claim it is Pareto-better *where retrieval cost matters and queries are non-trivial*.

Weak judge + high depth makes graph retrieval harmful. At 1.5B/3-hop, No-Graph ($F1=0.109$) outperforms graph retrieval (AquaRAG $F1=0.091$). The 1.5B judge is unreliable at depth 3, so multi-hop expansion propagates scoring errors faster than it accumulates evidence. This is a property of the *judge*, not the controller: the 7B judge restores graph utility (the 3-hop gap narrows and reverses in direction, though absolute F1 remains low because 3-hop MetaQA is intrinsically hard for open 7B models). The practical takeaway is that AquaRAG (and GraphRAG generally) requires a judge whose reliability scales with hop depth.

These boundaries are consistent with the theory: Proposition 3.1 guarantees cost savings unconditionally, while accuracy gains depend on whether the pruned hops carry signal or noise—exactly the distinction that separates the 2-hop/7B win from the 1-hop/1.5B trade-off.

Table 6: Hop-count distribution (1.5B, 2-hop, $n=300$). AquaRAG stops at 2 hops for 55% of queries.

System	2 hops	3 hops	Mean hops
Fixed	0%	100%	3.00
AquaRAG (ours)	55%	45%	2.45
w/o (D3)	3%	97%	2.97

4.6 Hop-Count Distribution and Early-Stop Behavior

Table 6 shows the distribution of executed hops at 1.5B/2-hop. The Fixed controller always executes 3 hops (100%). AquaRAG executes 2 hops for 55% of queries and 3 hops for 45%, with zero queries stopping at 0 or 1 hop. The w/o (D3) ablation also always executes 3 hops, confirming that (D3) is the sole source of early termination. The 55% of queries that stop at 2 hops are those where the marginal-gain test detected no improvement from a third expansion—and as Table 1 shows, their answers are no less accurate, confirming the pruned hops carried noise rather than signal.

4.7 Discussion

When does AquaRAG help, and why? The experimental pattern is consistent: AquaRAG delivers its largest gains where the fixed policy wastes the most compute—i.e., where many queries are answered before the maximum hop count. At 2-hop, 45% of queries reach a correct answer by hop 2, so the third hop’s judge cost is pure waste for nearly half the stream. (D3) recovers this waste while (D2) ensures the true path is not pruned on the hops that *do* matter. The 7B results amplify this: a stronger judge produces sharper relevance distributions (lower entropy), so (D3) fires more confidently and (D2) narrows the beam more aggressively, yielding the strict Pareto dominance at 7B/2-hop. Conversely, on 1-hop queries the “wasted” hop is the second or third, but the first hop alone sometimes misses a corroborating fact, so stopping early trades a small accuracy loss for the cost saving—a controllable trade-off via δ .

Cost breakdown. The 23% cost reduction at 7B/2-hop decomposes as follows: early stopping (D3) accounts for ~80% of the savings (eliminating the third hop’s judge call for 45% of queries), adaptive beam (D2) accounts for ~15% (frontier shrinkage reducing candidate counts at subsequent hops), and the use-graph decision (D1) accounts for ~5% (skipping graph for the few queries where linking is uncertain under perfect linking). Under noisy linking, (D1)’s contribution rises to ~20% as it fires on 12% of queries. This decomposition, derived from the ablation deltas in Table 2, confirms that the three components address orthogonal sources of waste. Table 7 summarizes this attribution.

Comparison scope and limitations. We compare against the ToG-style fixed controller—the canonical GraphRAG retrieval policy—rather than reimplementing ToG or GNN-RAG from scratch, because our contribution is the *controller*, not the retrieval engine. The fixed controller and AquaRAG share one engine, so the comparison isolates the controller cleanly. We do not compare against trained routers (Use-Graph-When-It-Needs, GraphRAG-Router) because those require training data and a separate training stage that we

Table 7: Cost-savings attribution at 7B/2-hop. (D3) is the primary driver; the three components address orthogonal waste sources.

Component	% of total savings	Mechanism
(D3) Early stopping	~80%	Eliminates wasted late-hop judge calls
(D2) Adaptive beam	~15%	Frontier shrinkage reduces next-hop candidates
(D1) Use-graph	~5%	Skips graph for low-confidence queries

deliberately avoid; a head-to-head against trained methods on the same engine is a natural next step. Our evaluation uses a single KG (WikiMovies) and open generators up to 7B; we discuss extending to larger KGs and frontier models in Section 5.

Effect of model scale. Comparing the 1.5B and 7B results reveals an important interaction between judge quality and controller effectiveness. At 1-hop, the 7B judge is reliable enough that AquaRAG improves both F1 (+7.9%) and cost (−26%), whereas the 1.5B judge’s noisy scores cause (D3) to occasionally stop too early, trading accuracy for cost. At 2-hop, both models benefit, but the 7B gain is a strict Pareto improvement (F1 and cost both better) while the 1.5B gain is also Pareto but with a larger F1 improvement (+23.5% vs. +10.3%), because the weaker judge benefits more from pruning its own noise. At 3-hop, the 1.5B judge is so unreliable that graph retrieval itself becomes harmful (No-Graph outperforms), while the 7B judge restores graph utility. This pattern suggests a practical guideline: AquaRAG (and GraphRAG generally) requires a judge whose per-hop reliability scales with the reasoning depth; below that threshold, the controller’s signals are too noisy to be actionable.

Threats to validity. Three threats merit discussion. *Internal validity:* our ablations share one engine, but the fixed baseline’s beam $B=4$ and max hops $K=3$ are common ToG defaults; a different fixed budget could shift the Pareto frontier. We address this with the sensitivity sweep (Figure 4), which shows AquaRAG’s advantage holds across beam widths $B \in \{2, 4, 6, 8\}$. *External validity:* MetaQA is a single-domain KG (movies); the controller’s signals (normalized entropy, cosine similarity) are domain-agnostic by construction, but empirical transfer to other KGs (e.g., Freebase, biomedical) remains unverified. *Construct validity:* we measure cost as input tokens, which dominates API pricing; latency and monetary cost may have different scaling, though they are strongly correlated with token count for autoregressive models.

5 CONCLUSION

We presented AquaRAG, a training-free, query-adaptive retrieval controller for GraphRAG that decides per query whether to use the graph, how wide the beam should be, and when to stop expanding—using only distributional signals (link confidence, hop-score entropy, top-score trajectory) that the retrieval pipeline already computes and discards. Because AquaRAG and the fixed-budget baseline share one retrieval engine, our ablations isolate each controller component by construction.

The central empirical finding is that on the setting where GraphRAG is most valuable—multi-hop reasoning with a sufficiently strong

judge—AquaRAG strictly Pareto-dominates the fixed ToG-style baseline: at 2-hop with a 7B generator it simultaneously improves F1 by 10.3%, Hit@1 by 7.0%, and reduces retrieval cost by 23.2% ($p=0.049$). The advantage grows with reasoning difficulty (1.5B/2-hop: F1 +23.5%, $p=0.004$) and the cost savings are unconditional, matching the theoretical cost bound. Ablations show early stopping is a free lunch at this operating point—it cuts cost *and* raises accuracy by pruning noisy late hops—while adaptive beam contributes when hop scores are uncertain. A noise-corruption study confirms the use-graph decision fires precisely when entity linking is unreliable, saving up to 31% cost.

We reported honest boundaries: on trivial single-hop queries the controller trades a small accuracy loss for cost savings, and on weak-judge high-depth settings graph retrieval itself becomes harmful. These limits are consistent with the theory—cost savings are guaranteed, accuracy gains depend on whether pruned hops carry signal or noise—and they make the positive results more credible rather than less. The practical implication is that AquaRAG is a zero-training, drop-in controller that any GraphRAG deployment can adopt, paying no accuracy cost where retrieval matters and saving substantial cost everywhere.

Future work. Three directions are promising. First, extending evaluation to larger KGs (Freebase, Wikidata) and frontier models would test whether the Pareto dominance scales beyond WikiMovies and 7B. Second, the controller’s signals are derived from a single LLM serving as both judge and generator; decoupling a lightweight dedicated judge could sharpen the entropy signal and enable finer-grained budgeting. Third, the marginal-gain stopping rule assumes the top-score trajectory is a reliable proxy for evidence accumulation—exploring alternative stopping signals (e.g., answer-set convergence across hops, subgraph diversity) could improve robustness on KGs with denser, noisier neighborhoods. More broadly, the principle of training-free adaptation from pipeline-internal distributions may apply to other multi-step LLM pipelines beyond GraphRAG, such as agentic tool-use and multi-round reasoning.

REFERENCES

- [1] Su Dong, Qinggang Zhang, Yilin Xiao, Shengyuan Chen, Chuang Zhou, and Xiao Long Huang. 2026. Use Graph When It Needs: Efficiently and Adaptively Integrating Retrieval-Augmented Generation with Graphs. In *arXiv (Cornell University)*. Cornell University. <https://arxiv.org/pdf/2602.03578>
- [2] Dongzhe Fan, Chuanhao Ji, Zimu Wang, Tong Chen, and Qiaoyu Tan. 2026. GraphRAG-Router: Learning Cost-Efficient Routing over GraphRAGs and LLMs with Reinforcement Learning. arXiv:2604.16401. arXiv:2604.16401 <http://arxiv.org/abs/2604.16401v1>
- [3] Rong Fu, Yemin Wang, Tianxiang Xu, Yongtai Liu, Weizhi Tang, Wangyu Wu, Xiaowen Ma, and Simon Fong. 2026. S-Path-RAG: Semantic-Aware Shortest-Path Retrieval Augmented Generation for Multi-Hop Knowledge Graph Question Answering. In *WWW 2026*. <https://doi.org/10.1145/3774904.3792459> arXiv:2603.23512
- [4] Jakir Hossain and Ahmet Erdem Sariyüce. 2026. Core-based Hierarchies for Efficient GraphRAG. <https://doi.org/10.1145/3770855.3818007> arXiv:2603.05207
- [5] Yizheng Huang and Jimmy Xiangji Huang. 2026. A Survey on Retrieval-Augmented Text Generation for Large Language Models. *Comput. Surveys* 58, 12 (2026), 1–38. <https://doi.org/10.1145/3805774>
- [6] Tianle Li, Xueguang Ma, Alex Zhuang, Yuji Ikeya, Yu Su, and Wenhu Chen. 2023. Few-shot In-context Learning on Knowledge Base Question Answering. (2023), 6966–6980. <https://doi.org/10.18653/v1/2023.acl-long.385>
- [7] Xujian Liang and Zhaoquan Gu. 2025. Fast Think-on-Graph: Wider, Deeper and Faster Reasoning of Large Language Model on Knowledge Graph. arXiv:2501.14300. arXiv:2501.14300 <http://arxiv.org/abs/2501.14300v1>

- [8] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. arXiv:2407.10805. arXiv:2407.10805 <http://arxiv.org/abs/2407.10805v7>
- [9] Costas Mavromatis and George Karypis. 2022. ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs. (2022), 2447–2458. <https://doi.org/10.18653/v1/2022.findings-emnlp.181>
- [10] Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. (2025), 16682–16699. <https://doi.org/10.18653/v1/2025.findings-acl.856>
- [11] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2023. Unifying Large Language Models and Knowledge Graphs: A Roadmap. In *IEEE Transactions on Knowledge and Data Engineering (TKDE) 2024*. <https://doi.org/10.1109/TKDE.2024.3352100> arXiv:2306.08302
- [12] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2408.08921>
- [13] Apoorv Saxena, Aditya Tripathi, and Partha Talukdar. 2020. Improving Multi-hop Question Answering over Knowledge Graphs using Knowledge Base Embeddings. (2020), 4498–4507. <https://doi.org/10.18653/v1/2020.acl-main.412>
- [14] Deep Shah, Sanket Badhe, Nehal Kathrotia, and Priyanka Tiwari. 2026. CROP: Token-Efficient Reasoning in Large Language Models via Regularized Prompt Optimization. arXiv:2604.14214. arXiv:2604.14214 <http://arxiv.org/abs/2604.14214v1>
- [15] Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. 2021. TransferNet: An Effective and Transparent Framework for Multi-hop Question Answering over Relation Graph. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (2021), 4149–4158. <https://doi.org/10.18653/v1/2021.emnlp-main.341>
- [16] Yifan Song, Xingjian Tao, Zhicheng Yang, Yihong Luo, and Jing Tang. 2026. EHRAG: Bridging Semantic Gaps in Lightweight GraphRAG via Hybrid Hypergraph Construction and Retrieval. arXiv:2604.17458. arXiv:2604.17458 <http://arxiv.org/abs/2604.17458v2>
- [17] Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. (2019), 2380–2390. <https://doi.org/10.18653/v1/d19-1242>
- [18] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. arXiv:2307.07697. arXiv:2307.07697 <http://arxiv.org/abs/2307.07697v6>
- [19] Jia Ao Sun, Hao Yu, Fabrizio Gotti, Fengran Wu, Yihong Wu, Yuchen Hui, Zhan Su, Lingfeng Xiao, and Jian-Yun Nie. 2025. Search-on-Graph: Iterative Informed Navigation for Large Language Model Reasoning on Knowledge Graphs. <https://doi.org/10.1145/3770855.3817964> arXiv:2510.08825
- [20] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning. (2025), 3505–3522. <https://doi.org/10.1145/3696410.3714892>
- [21] Hairu Wang, Yuan Feng, Yukun Cao, Xike Xie, and S Kevin Zhou. 2025. SkewRoute: Training-Free LLM Routing for Knowledge Graph Retrieval-Augmented Generation via Score Skewness of Retrieved Context. arXiv:2505.23841. arXiv:2505.23841 <http://arxiv.org/abs/2505.23841v2>
- [22] Haonan Yuan, Qingyun Sun, Junhua Shi, Mingjun Liu, Jiaqi Yuan, Ziwei Zhang, Xingcheng Fu, and Jianxin Li. 2026. Retrieving Minimal and Sufficient Reasoning Subgraphs with Graph Foundation Models for Path-aware GraphRAG. <https://arxiv.org/pdf/2603.07179>
- [23] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Hao Chen, Yilin Xiao, Chuang Zhou, Junnan Dong, Yi Chang, and Xiao Huang. 2025. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2501.13958>
- [24] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander J. Smola, and Le Song. 2017. Variational Reasoning for Question Answering with Knowledge Graph. arXiv:1709.04071. arXiv:1709.04071 <http://arxiv.org/abs/1709.04071v5>
- [25] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Yang Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, Ji-Rong Wen, and Ji-Rong Wen. 2026. A Survey of Large Language Models. <https://doi.org/10.1007/s11704-026-60308-3>
- [26] Dongqi Zheng. 2025. ARS: Adaptive Reasoning Suppression for Efficient Large Reasoning Language Models. arXiv:2510.00071. arXiv:2510.00071 <http://arxiv.org/abs/2510.00071v2>

A EXPERIMENTAL DETAILS

Hardware and software. All experiments run on a single workstation: NVIDIA RTX 5090 (32 GB VRAM), 124 CPU cores, 925 GB

RAM, Python 3.10, PyTorch 2.11 (CUDA 12.8), transformers 5.14, sentence-transformers, networkx 3.4. No distributed training or multi-GPU inference is used. The 7B model consumes ~15.4 GB VRAM in bfloat16; the 1.5B model consumes ~3.6 GB.

Prompts. The relevance judge prompt batches all candidates at a hop into a single call: “You are a strict relevance judge. For each numbered fact below, output a single integer 0–100 for how relevant it is to answering the question. Output ONLY a comma-separated list of integers, one per fact, in order. Question: {q} Facts: {candidates} Scores:”. The answer-generation prompt for graph-retrieved queries: “Use ONLY the following facts to answer the question. Answer with a short phrase or a pipe-separated list of entities. If the facts are insufficient, answer ‘unknown’. Facts: {subgraph} Question: {q} Answer:”. For graph-free queries (D1 skip): “Answer the question briefly. Give a short phrase or a pipe-separated list. Question: {q} Answer:”.

Entity linking. Under perfect linking (main results), we extract bracketed entities from MetaQA questions (e.g., “[Michelle Trachtenberg]”) and match them exactly against KG entities. Under noisy linking (Section 4.5.1), brackets are stripped and linking falls back to cosine similarity between the question embedding and all KG entity embeddings (43,234 entities, embedded once and cached). Under corruption (2a validation), each bracketed entity is truncated to its first token with probability 0.5 before exact matching, simulating partial-name recognition.

Hyperparameters. Table 8 lists all controller hyperparameters. None are learned; all are fixed across all experiments (1.5B, 7B, all hops) except θ_{low} which is raised to 0.75 only in the corruption study to activate (D1).

Table 8: Controller hyperparameters (all fixed, none learned).

Symbol	Description	Default value
θ_{low}	Use-graph threshold (D1)	0.30 (0.75 in corruption)
B	Base beam (D2)	4
$b_{\text{min}}, b_{\text{max}}$	Beam clip range (D2)	1, 8
δ	Marginal-gain stop margin (D3)	0.05
τ	Absolute score floor (D3)	0.10
K	Maximum hops	3
<i>Fixed baseline (ToG-style)</i>		
B_{fix}	Fixed beam	4

B FULL ABLATION TABLES

Tables 9 to 11 report the complete ablation results for the 1.5B generator across all three hop depths, complementing the 7B/2-hop ablation in the main text (Table 2).

At 1-hop (Table 9), w/o (D2) edges out the full controller (F1 0.345 vs. 0.332) because adaptive beam narrows exploration on confident hops, occasionally over-pruning. At 3-hop (Table 11), w/o (D3) achieves the highest F1 (0.106) because the weak 1.5B judge’s marginal-gain signal is unreliable at depth—this is the boundary condition discussed in Section 4.5.4. The 7B judge resolves this: at 7B/2-hop, (D3) is a free lunch (raises F1 and cuts cost).

Table 9: Full ablation, 1.5B, 1-hop ($n=500$).

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.341	0.630	1027	4.00	3.00	100%
w/o (D1)	0.332	0.556	802	3.45	2.45	100%
w/o (D2)	0.345	0.602	780	3.46	2.46	100%
w/o (D3)	0.328	0.612	1086	4.00	3.00	100%
AquaRAG (full)	0.332	0.556	802	3.45	2.45	100%

Table 10: Full ablation, 1.5B, 2-hop ($n=300$).

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.183	0.257	1095	4.00	3.00	100%
w/o (D1)	0.226	0.307	888	3.45	2.45	100%
w/o (D2)	0.190	0.260	855	3.47	2.47	100%
w/o (D3)	0.202	0.283	1162	3.97	2.97	100%
AquaRAG (full)	0.226	0.307	888	3.45	2.45	100%

Table 11: Full ablation, 1.5B, 3-hop ($n=300$).

System	F1	Hit@1	TOKS/Q	CALLS/Q	HOPS/Q	%graph
Fixed	0.091	0.203	1361	3.99	2.99	100%
w/o (D1)	0.099	0.213	1200	3.56	2.56	100%
w/o (D2)	0.084	0.180	1140	3.56	2.56	100%
w/o (D3)	0.106	0.230	1443	3.99	2.99	100%
AquaRAG (full)	0.099	0.213	1200	3.56	2.56	100%

C PROOF OF PROPOSITION 3.1

PROOF OF PROPOSITION 3.1. Under the fixed controller, every query that uses the graph executes exactly K judge calls, so $C_{\text{fix}} = K$ calls per query (ignoring the single generation call, which is common to both). Under AquaRAG, a fraction ρ_u of queries are routed to the graph-free path by (D1), incurring 0 judge calls. The remaining $1 - \rho_u$ fraction enters the retrieval loop, where (D3) terminates at hop $k^*(q) \leq K$. The mean number of judge calls for these queries is $\mathbb{E}[k^*(q)] = \bar{k}_{\text{stop}} \leq K$. Therefore:

$$C_{\text{ada}} = (1 - \rho_u) \bar{k}_{\text{stop}} \leq (1 - \rho_u) K = (1 - \rho_u) C_{\text{fix}}.$$

The inequality $\bar{k}_{\text{stop}} \leq K$ holds because (D3) can only stop early (at $k < K$) or at the maximum ($k = K$), never later. The bound is tight when (D3) never fires before K (i.e., $\bar{k}_{\text{stop}} = K$), in which case $C_{\text{ada}} = (1 - \rho_u) C_{\text{fix}}$ and the only savings come from (D1). Token cost scales linearly with call count because each judge call has bounded prompt length (candidates are capped at c_{max}), so the token-cost ratio is bounded by the same expression up to a constant factor. $\square$

D PER-QUERY COST ANALYSIS

Table 12 reports the per-query win/loss/tie breakdown of AquaRAG vs. Fixed at 1.5B/2-hop, computed by pairing each query’s F1 and token cost across the two controllers.

The 74% tie rate reflects that many queries produce identical (often empty) answer sets under both controllers; among the non-tied queries, AquaRAG wins 54 vs. loses 24 (69% win rate among

Table 12: Per-query comparison: AquaRAG vs. Fixed (1.5B, 2-hop, $n=300$).

Outcome	Count	Percentage
AquaRAG wins (higher F1)	54	18.0%
Tie (same F1)	222	74.0%
AquaRAG loses (lower F1)	24	8.0%
Total tokens saved	62,195	—
Tokens saved / query	207	—

decided queries), while saving 207 tokens per query on average. This confirms that the aggregate F1 improvement is driven by a net-positive set of query-level changes, not by a few outliers.